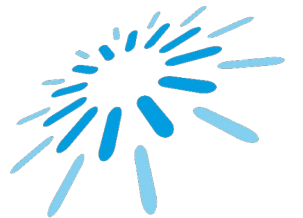




Software Architecture and Design Patterns

Prof. Dr. Jóakim von Kistowski

Lecture 08



TH Aschaffenburg
university of applied sciences

	Monday	Tuesday	Wednesday	Thursday	Friday
08:00 - 09:30					Vorlesung Software Design
09:45 - 11:15					Übung Software Design
11:30 - 13:00			Lecture Software Design International		
	Lunch Break				
14:00 - 15:30			Exercises Software Design International		
15:45 - 17:15					
17:30 - 19:00					

What is the factory pattern?

What is the observer pattern?

Introduction to Macro-Architecture



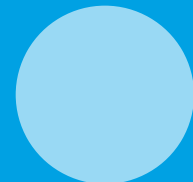
Recap Macro-Architecture and Conway's Law

Software Components

Architectural Styles

Architectural Patterns

Concluding Remarks



TH Aschaffenburg
university of applied sciences

What is Conway's law?

Recap: Software Architecture and People

- Conway's Law

„Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization's communication structure.“

Melvin E. Conway

Human interactions, relations and organizational structures are a part of software architecture

Question

Why is Conway's law important in interface design?

Task – ADR

Decision: Multiple Apps

Problem: Our bank is considering splitting of our banking app into three different Apps:
(1) TAN/Login, (2) Bank-Account-Management, (3) Stocks

Environmental Conditions: The existing app already exists and has 1 million users. Our Bank has 3000 employees.

Solution Alternative 1: A Single App

Solution Alternative 2: Three Apps

Quality Objectives: (1) Security, (2) Maintainability, (3) Availability

Make the Decision. Explicitly consider the effect of Conway's Law!

What is Macro-Architecture?

Recap: Macro Software Architecture

Architecture dealing with large scale decisions

- concerning multiple software systems
- concerning multiple teams
- concerning organizational structure and its interplay with technology
- Examples:
 - Designing the overarching architecture for a software system developed by > 50 people
 - Designing architectural guidelines and rules for an entire company

Introduction to Macro-Architecture



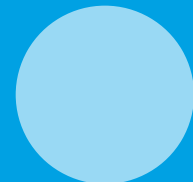
Recap Macro-Architecture and Conway's Law

Software Components

Architectural Styles

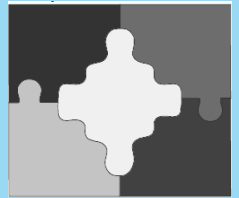
Architectural Patterns

Concluding Remarks



Software Components – Definition

- The idea of composing software systems from reusable components is more than 40 years old
(McIlroy 1968, NATO-Conference Garmisch-Patenkirchen)
- “A software component is a **unit of composition** with contractually specified interfaces and explicit context dependencies. It **can be deployed independently** and is subject to composition by third parties.” [WCOP 96]
 - has provided (and required) interfaces
 - hides its implementation
 - is self-contained
 - adheres to a component model

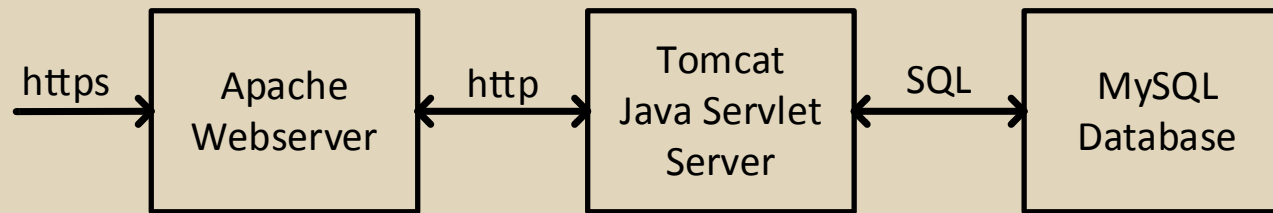


Software Components – Informal Definition

- A component is a **contractually specified building block** for software that can be **readily used by third parties *without understanding*** its internal structure
- Key Points
 - Readily used: effort for deployment, assembly, composition or adaptation should be as low as possible
 - Black-box principle → can be used by third parties without understanding its internal structure
 - Information on component's internals may still be provided for tools

Question

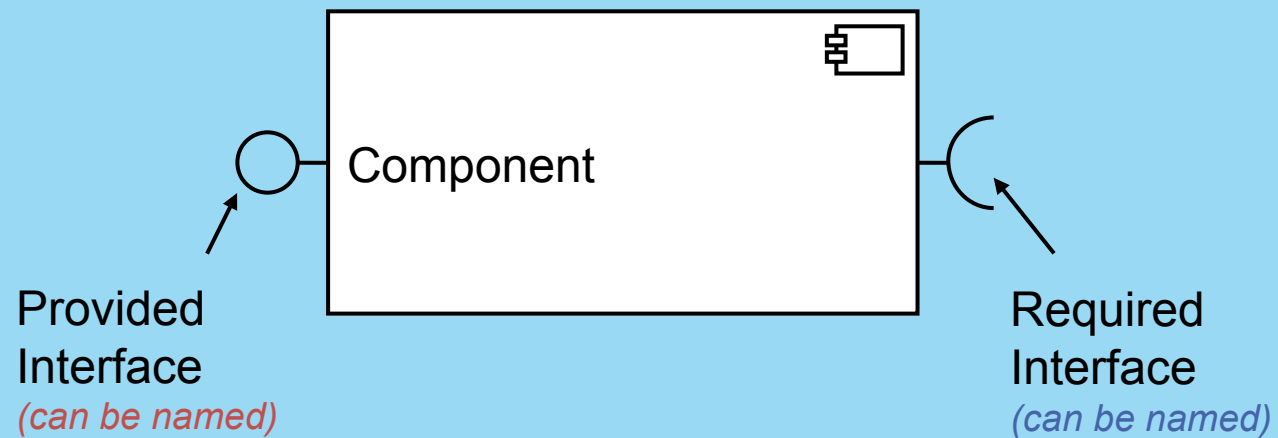
Are these Components?



Criteria

- has provided (and required) interfaces (✓)
- hides its implementation ✓
- is self-contained ✓
- adheres to a component model ✗

Component Notation (UML 2)



Introduction to Macro-Architecture

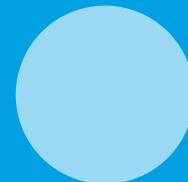
Recap Macro-Architecture and Conway's Law

Software Components

Architectural Styles

Architectural Patterns

Concluding Remarks



Architectural Styles

- The architecture of a system may conform to a generic style
 - A style is a set of **constraints** that apply system wide
- Styles can simplify the problem of defining system architectures
 - Help to avoid reinventing the wheel
 - However, large systems are heterogeneous and do not follow a single style
- Architectural styles may have different application areas
- *“An architectural style is a coordinated set of **architectural constraints** that restricts the roles/features of **architectural elements** and the **allowed relationships** among those elements within any architecture that conforms to that style.”* [Fielding]

Architectural Styles – Examples

Category	Styles
Structure	(object-oriented), layered, component-based, microkernel, pipeline / pipe-and-filter
Deployment	client/server, N-tier, three-tier, peer-to-peer
Communication / Distribution	service-oriented (SOA), message-oriented / message bus / publish-subscribe, event-based / event-driven, space-based

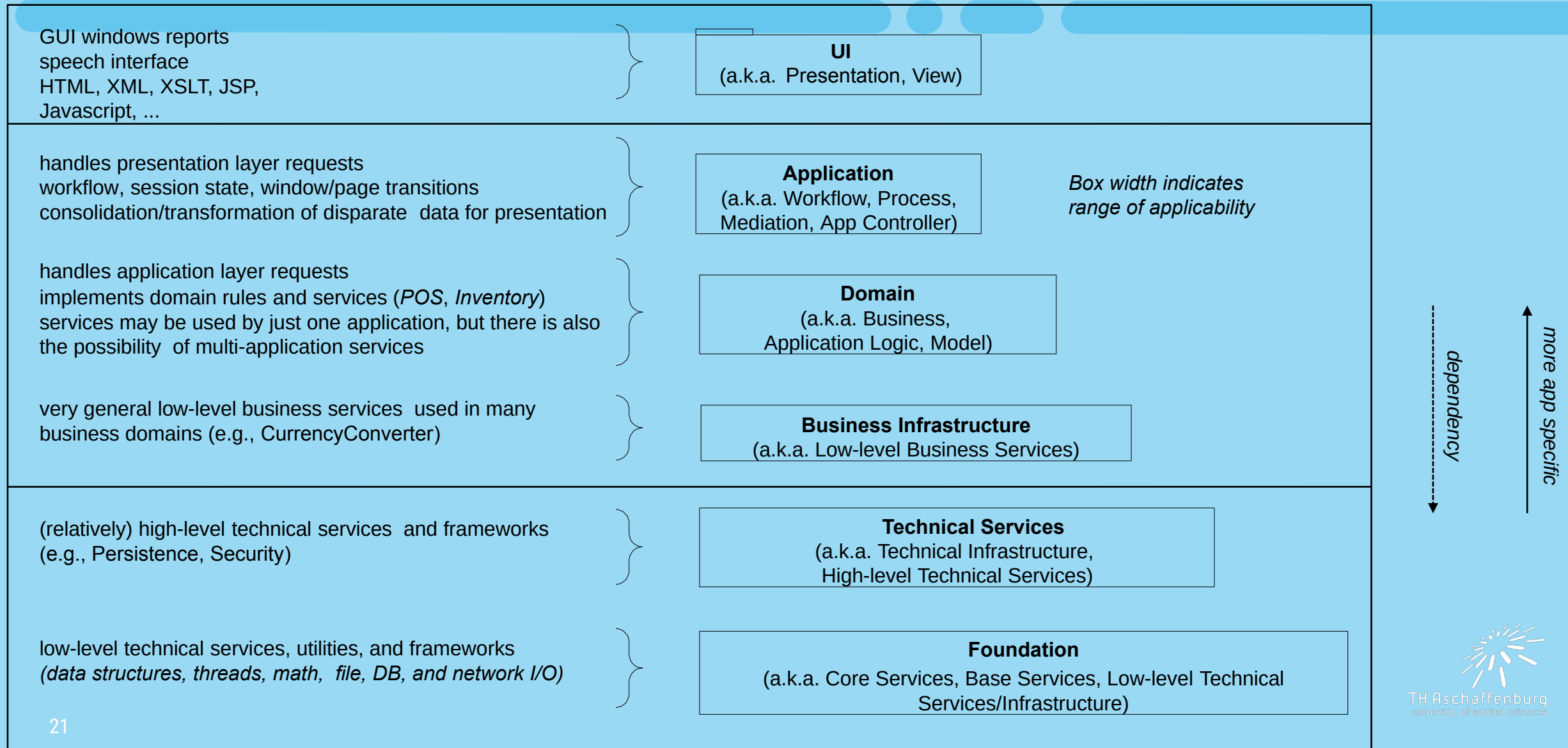
[<http://msdn.microsoft.com/en-us/library/ee658117.aspx>]

- In practice, there is often no clear separation-of-concerns
 - Example: microservice architecture (more later)
 - even the term “Architectural Style” itself is still widely discussed
- Thus, take the above examples with a grain of salt
 - as they mix logical, deployment, and control decisions

Layered Architecture: Typical System Layers

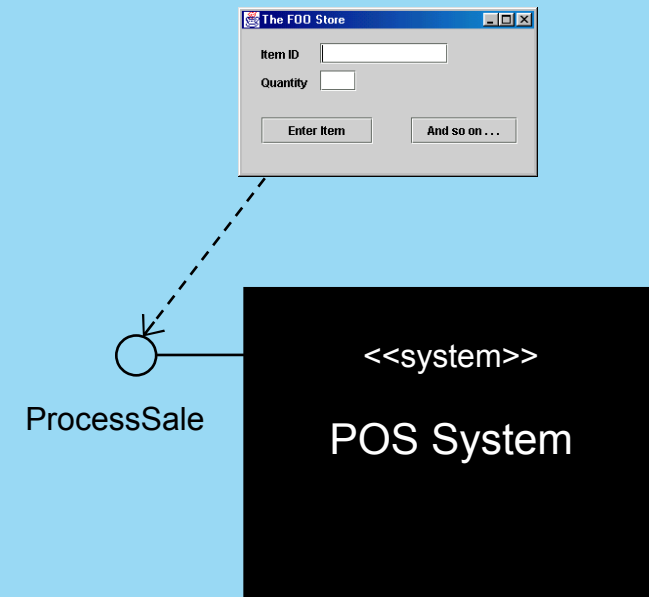
- **User Interface (UI) / Presentation Layer**
 - Responsible for managing *interactions with the user*
 - i.e., data presentation, user input, screen flow
- **Application (Business) Logic and Domain (Object) Layer**
 - Implement system operations, workflow, and session management
 - Include software objects representing domain concepts (e.g., a design class Sale) that *fulfill application requirements*, such as calculating a sale's total
 - Not to be confused with the Domain Model
- **Technical Services / Data Link Layer**
 - General purpose objects and subsystems that provide supporting technical services, (e.g., interfacing with a database or error logging)
 - Usually application independent & reusable across multiple systems

Layered Architecture [Larman]



Application Layer (Service Layer in [Fowler])

- Part of the system that **handles the incoming requests** and is responsible for:
 - implementing the system operations (e.g., ProcessSale in the POS example)
 - remembering **session state** and **controlling the flow of work** (i.e., by controlling the order of windows or web pages)
- In very small systems, it is optional to have an application layer
 - i.e., domain objects can be directly called by UI
 - however, for multi-tier architectures it is usually mandatory
 - due to more complex session handling



Domain Layer

- Contains the domain objects and the application / business logic
 - inspired by the domain model
 - this is where object design becomes necessary
 - and object-oriented programming is applied
- Typically very application specific
 - → domain objects should be kept in the domain layer

handles presentation layer requests
workflow, session state, window/page transitions
consolidation/transformation of disparate data for presentation

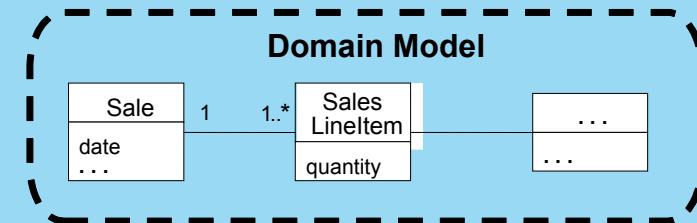
handles application layer requests
implements domain rules and services (*POS*, *Inventory*)
services may be used by just one application, but there is also the possibility of multi-application services

23
very general low-level business services used in many

Application
(a.k.a. Workflow, Process,
Mediation, App Controller)

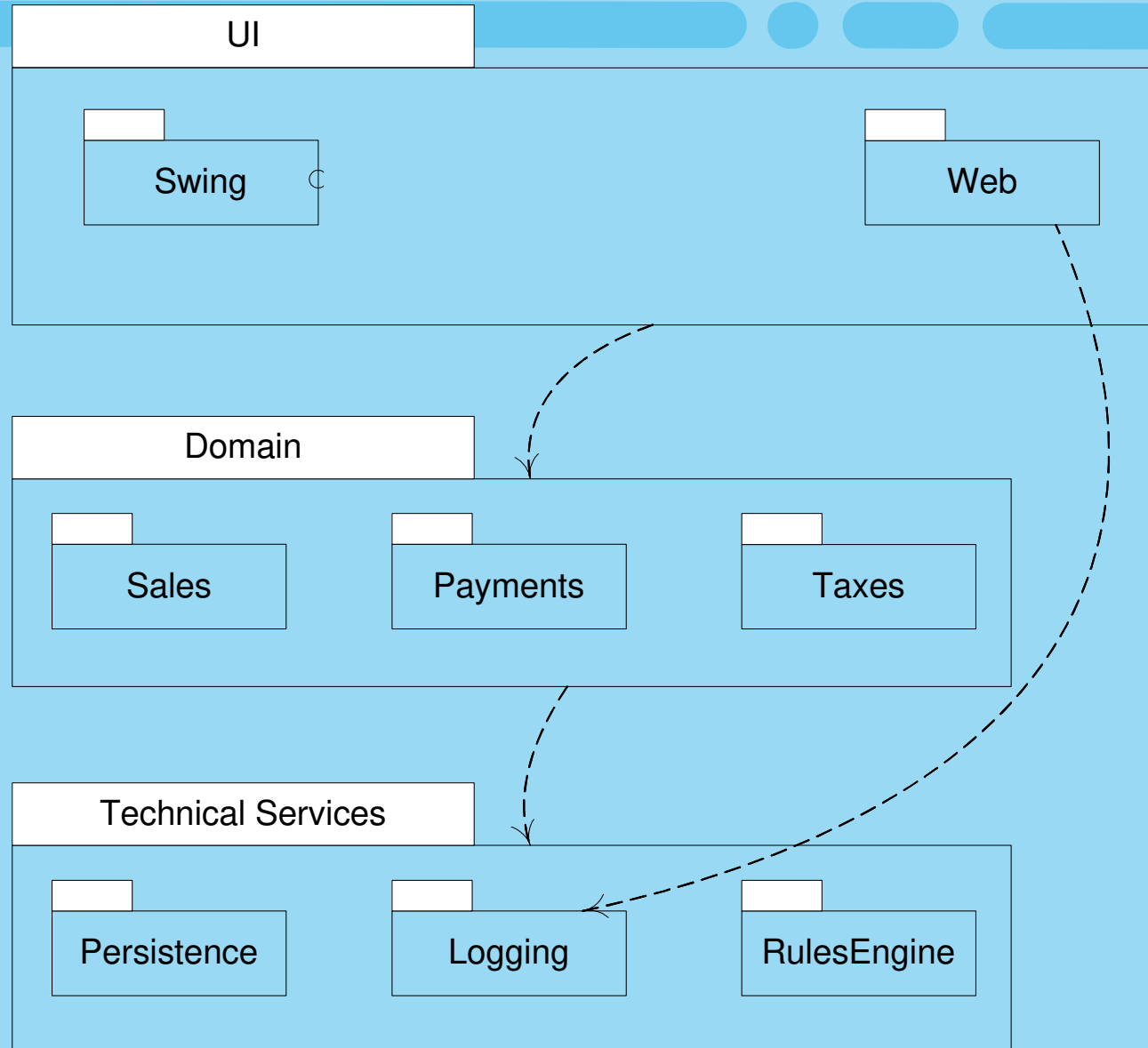
Domain
(a.k.a. Business,
Application Logic, Model)

Business Infrastructure



inspires

Example: Layers in Package Diagram Notation



Question

Which of these distinctions of modular programs over non modular are true?

a) They are more complex to understand and explain

b) They are more difficult to document

c) They are easier to change

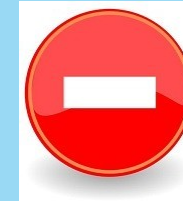
d) Testing and debugging is more complex (comparatively)

Layered Architecture: Pros and Cons



Advantages

- ✓ Simple to implement
- ✓ Abstraction due to separation of layers
- ✓ Isolation between layers protects against modification consequences
- ✓ Low coupling



Disadvantages

- Scalability
- Data flow through each layer causes the **Sinkhole Problem**

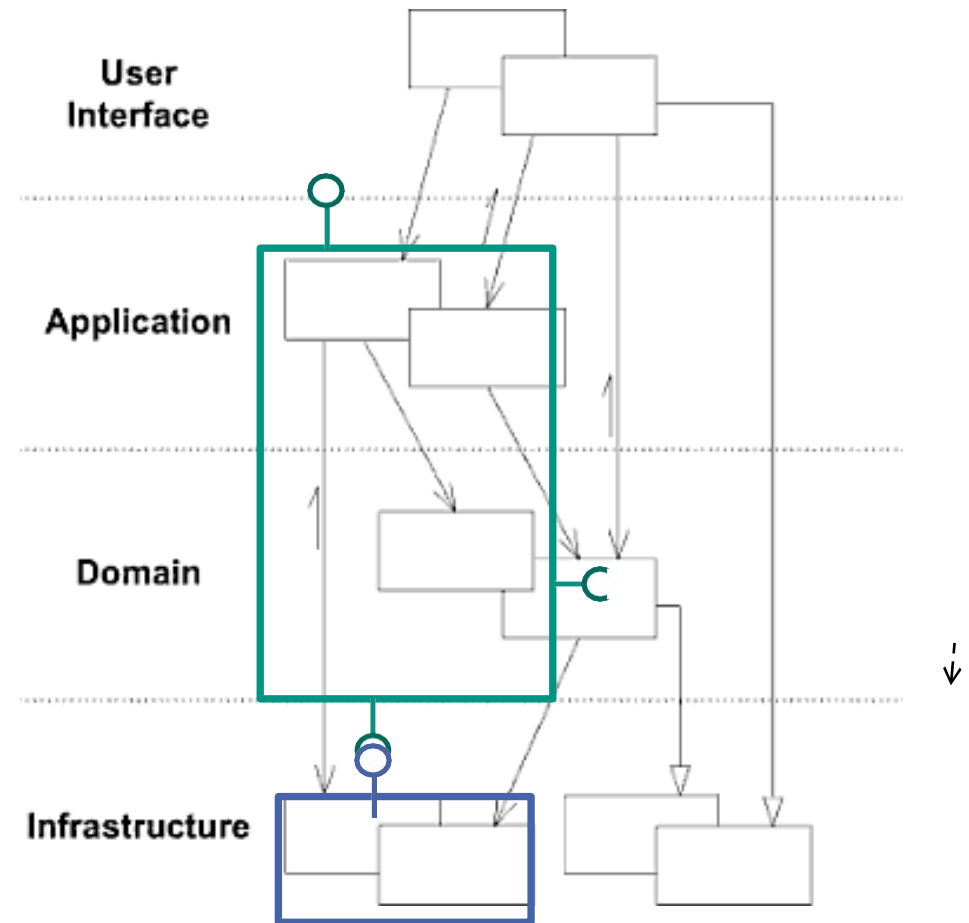
Question

**Regarding Conway's law:
How might a layered architecture influence people
and organizations?**

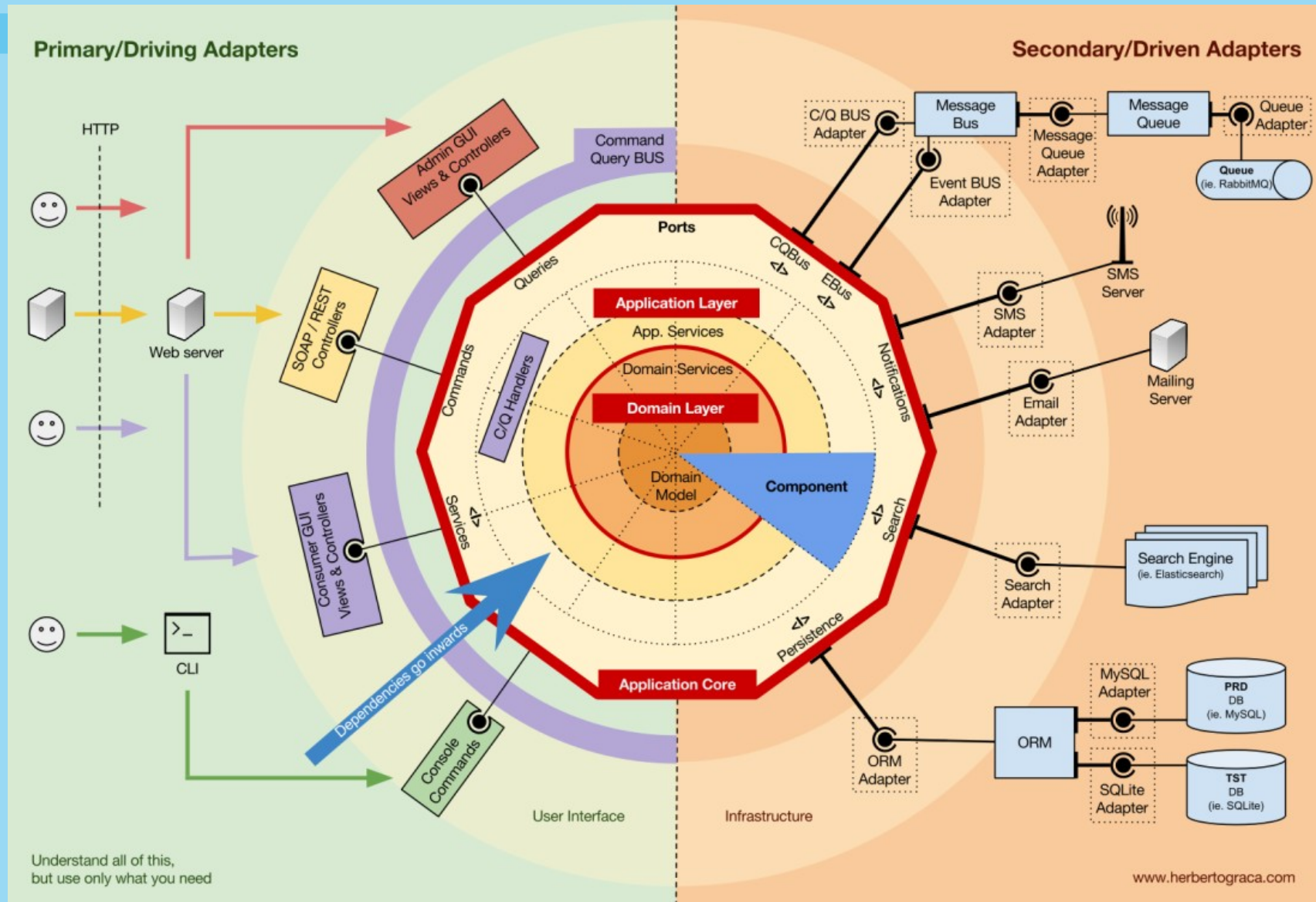
Software Components vs. Layers

Components are typically “feature-oriented”

- deliver some well defined chunk of functionality
- typically partition the functionality within a layer and may possibly crosscut through multiple layers

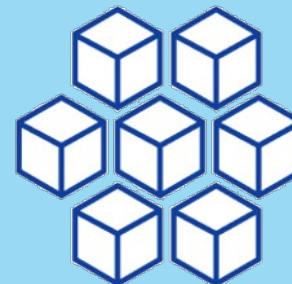
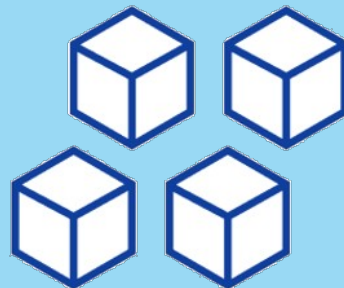
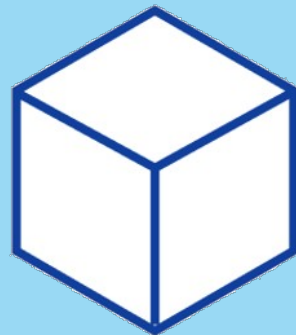


Example: Components in a Layered Architecture



Monolithic vs. Distributed Architectures

- **Monolithic:** The whole system is bundled into one deployable unit
 - **Pro:** Simple architecture
 - **Con:** Single point of failure, limited room for scaling
- **Distributed:** The system is deployed into multiple (independent) units communicating over a network
 - **Pro:** Scalability, elasticity, robustness, and availability
 - **Con:** More complex and challenging problems than monolithic architectures
 - **Con:** Heavily dependent on the offered reliability, latency, bandwidth, cost, security, and topology of the underlying network

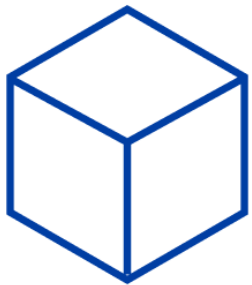


(Classic) Distributed Architectures

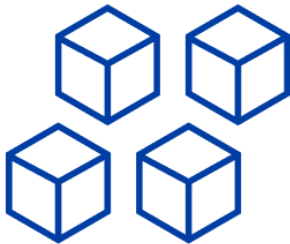
- Client-server architectures
 - Structuring of a system that partitions tasks or workloads between the providers of a resource or service, called **servers**, and service requesters, called **clients**
- Multi-tier architectures
 - Structuring of a system into **tiers** that implement distinct features such as the presentation logic or the application logic
 - Clear separation of responsibilities between tiers (typically include client-server relationships)
- Peer-to-peer architectures
 - System consists of interconnected **peers** of similar capabilities and responsibilities, where the peers can act as both servers and clients
 - Designed to be self-organizing and to scale well with the number of participating peers
- Distributed object architectures (e.g., CORBA, RMI)
 - any object on the system may provide and use services from other objects
 - no distinction between client and server objects

Modern Distributed Architectures

- Service-oriented Architectures (SOA)
- Microservice Architectures
- Cloud-native Architecture



MONOLITHIC
Single unit

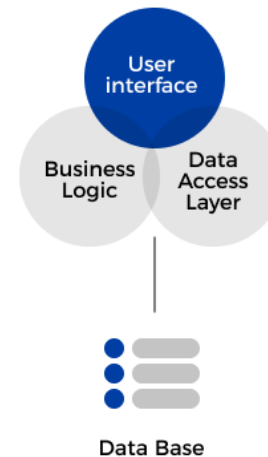


SOA
Coarse-grained
(Service-oriented
Architecture)

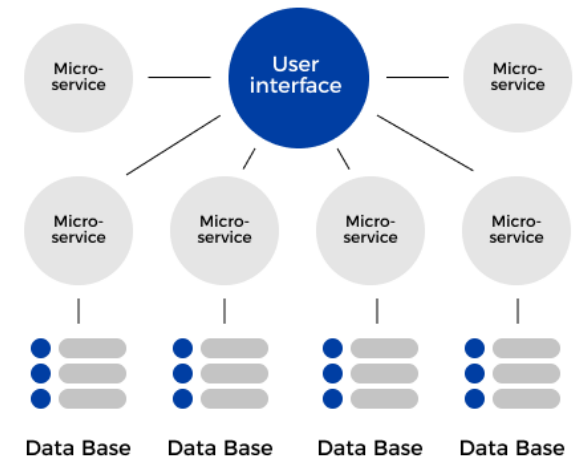


MICROSERVICES
Fine-grained

MONOLITHIC ARCHITECTURE



MICROSERVICE ARCHITECTURE



We'll talk about those in later lectures.

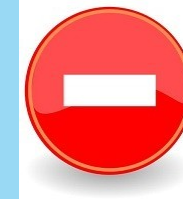
Source: <https://divante.com/blog/monolithic-architecture-vs-microservices/>

Distributed Systems: Pros and Cons (On paper, missing the most important aspect)



Advantages

- ✓ resource sharing
- ✓ openness
- ✓ concurrency
- ✓ scalability
- ✓ fault tolerance
- ✓ transparency



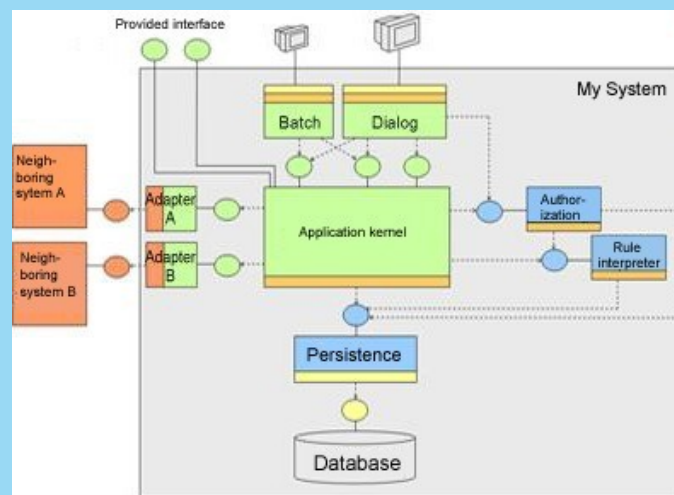
Disadvantages

- complexity
- security
- manageability
- unpredictability
- testability

**Regarding Conway's law:
How might a distributed architecture influence
people and organizations?**

Reference Architectures

- The system architecture may conform to a generic reference model
 - Derived from a study of **the application domain**
 - Acts as a standard against which systems can be built and evaluated
- Reference architecture models can be used as **templates**
 - They are more concrete and can be seen as instantiations of styles in a specific context



sd&m's Quasar

Introduction to Macro-Architecture

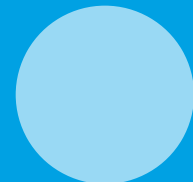
Recap Macro-Architecture and Conway's Law

Software Components

Architectural Styles

Architectural Patterns

Concluding Remarks



Recap: Software Patterns

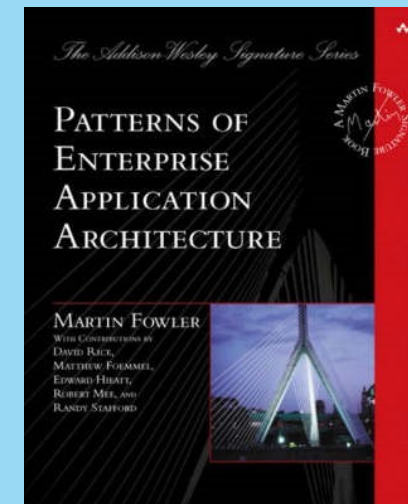
- In general, patterns are recurring solutions for recurring problems
- Software development knows a variety of patterns –
 - enterprise patterns
 - architectural patterns [Fowler]
 - (object) design patterns
 - General Responsibility Assignment Software Patterns (GRASP) [Larman]
 - Gang of Four Design Patterns [GoF]
 - idioms (sometimes identical to coding guidelines)
 - analysis patterns
- A pattern always needs to be adapted for a specific purpose
“you have the responsibility and the fun of completing it” [Fowler]



Our focus
today

Architectural Patterns

- Many of the arch. aspects discussed so far have been codified in **patterns**
- Boundary btw. arch. patterns and object design patterns is sometimes blurry
 - As many patterns can be used on both levels
 - As a rule of thumb: as soon as a pattern **crosses the boundaries of architectural elements**, it can be seen as an architectural pattern
- Groups of architectural patterns are related to –
 - domain / business logic
 - data sources and object/relational (O/R) mapping
 - (web) presentation and session handling
 - distribution and concurrency
 - basic issues
- Some architectural patterns target specific architectural styles only (e.g. Micro-Service patterns)



Question

Which of the concepts “architectural style”, “reference architecture”, or “architectural pattern” are described in (1), (2), and (3)?

- 1) Solution to recurring problem where several forces have to be balanced**
- 2) Cross-cutting principles (e.g., object-oriented, modular), independent of application**
- 3) Defines domain concepts, components, and subsystems that can be used as a template for applications in a given domain**

a) 1. Style, 2. Pattern,
3. Ref. Arch.

b) 1. Pattern, 2. Style,
3. Ref. Arch.

c) 1. Ref. Arch., 2. Style,
3. Pattern

d) 1. Ref. Arch., 2. Pattern,
3. Style

Terminology

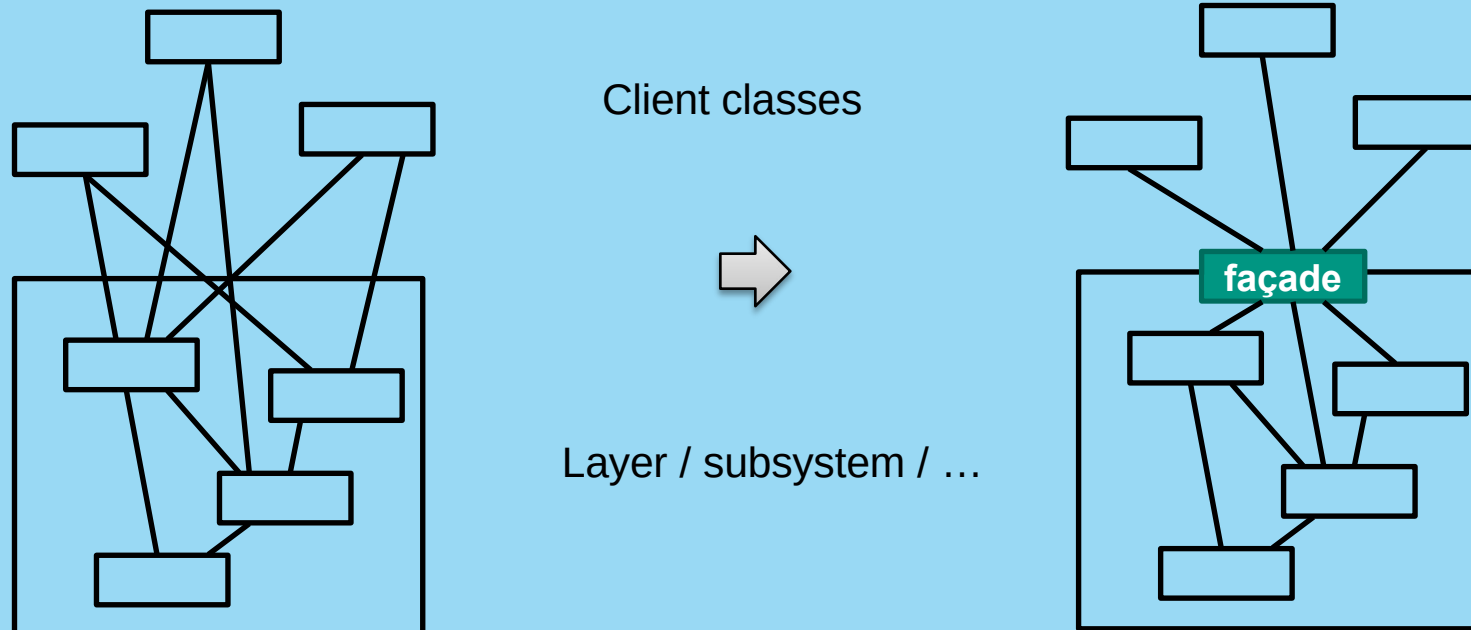
A word of warning!

- Software architecture is a relatively new field where many terms are still overloaded
- **Architectural Style**
 - Cross-cutting principles (e.g., object-oriented style, modular style)
 - Independent of application domain
 - Often synonymously (and incorrectly) used for „architectural pattern“
- **Reference Architecture**
 - Defines domain concepts, components, and subsystems that can be used as a template (specific to a given application domain)
- **Architectural Pattern**
 - Solution to recurring situation where several forces have to be balanced



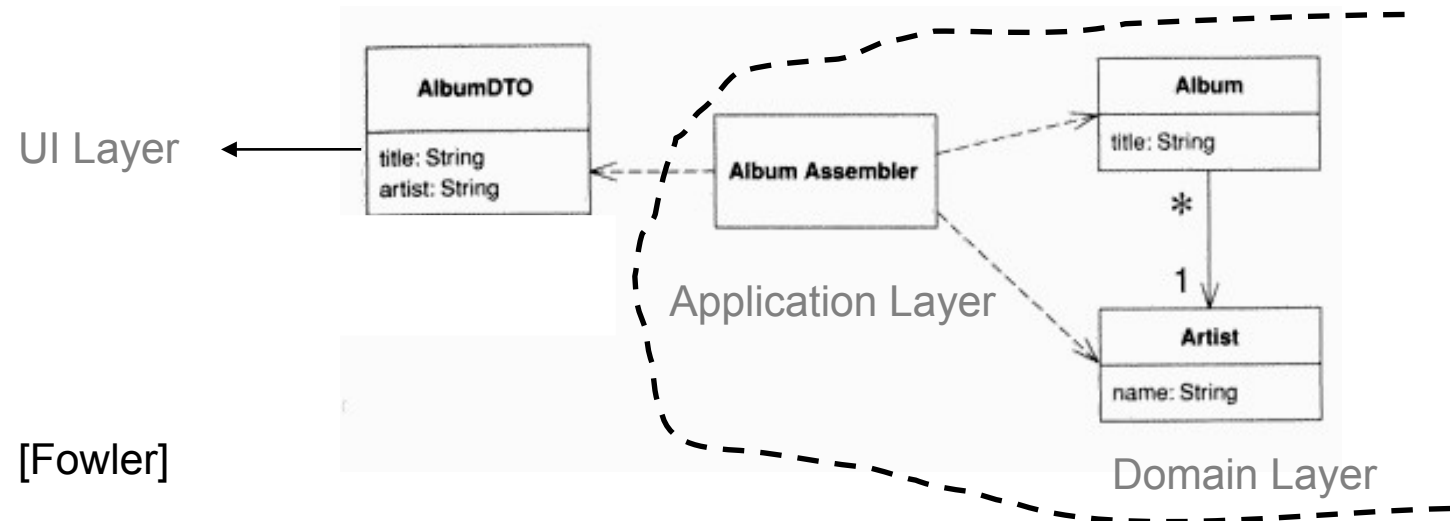
Façades

- Layers sometimes contain a large number of classes (or modules/components/services)
 - Each layer can also be seen as a subsystem
- To reduce coupling between layers, **façade classes** are often used
- A façade is a general concept that can be applied to any software element such as a layer, a subsystem, a component, a package, etc.



Data Transfer Object (DTO)

- An object that carries data between processes or architectural elements (i.e., layers, components, tiers, etc.)
- in order to reduce the number of method calls
 - i.e., to reduce coupling between architectural elements
 - it usually contains only attributes and getter and setter methods for them
- Domain objects usually cannot be passed directly due to complex dependencies



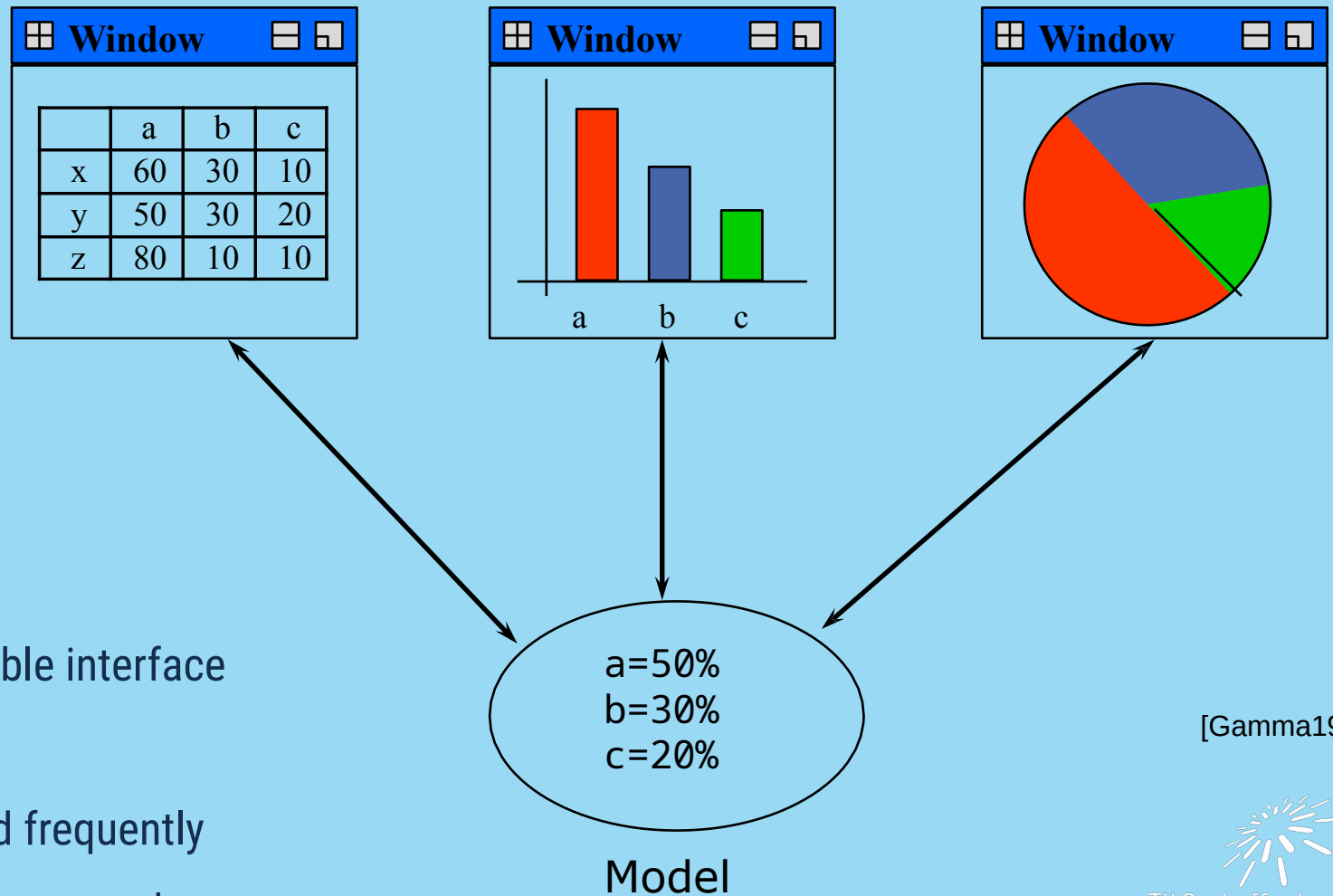
Model-View Separation Principle (1/2)

- One of the core tenets of good software engineering is separating UI/presentation logic from domain/application logic, since –
 - They deal with different concerns
 - Use different libraries, skills, etc.
 - It allows to create different UI views for an application
 - e.g., HTML, command line, PC vs. Handheld device (smartphone /tablet)...
 - Testing UI objects is usually hard
 - Model-view separation facilitates testing of the application core
- Including appl. logic in the UI is known as the **Smart UI Anti-Pattern**
- Separation of control logic and UI is also recommended
→ **Model-View-Controller Pattern**

Model-View Separation Principle (2/2)

- More specifically
 - Model → Domain layer objects
 - View → UI objects, such as windows, Web pages, ...
- **Part 1:** Do not connect or couple non-UI objects directly to UI objects
 - E.g., don't let a Sale object (a non-UI domain layer object) have a reference to a UI window object
- **Part 2:** Do not put application logic (such as a tax calculation) in the UI object methods
 - UI objects should only initialize UI elements, receive UI events (such as a mouse click on a button), and delegate requests for application logic on to non-UI objects (such as domain layer objects)

Model View Controller (MVC) Pattern (1/2)



[Gamma1995]

Context

- Interactive applications with a flexible interface

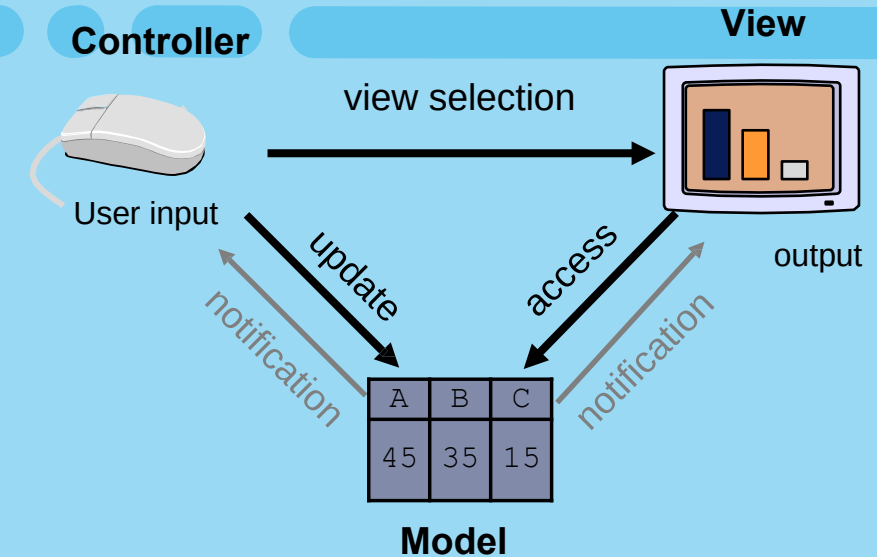
Problem

- User interfaces need to be changed frequently
- Different user interfaces must be supported

Model View Controller (MVC) Pattern (2/2)

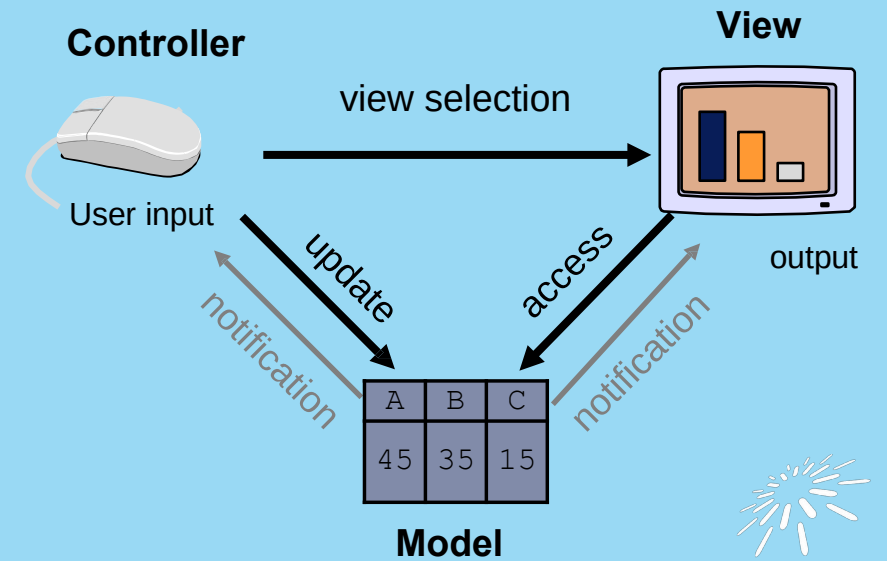
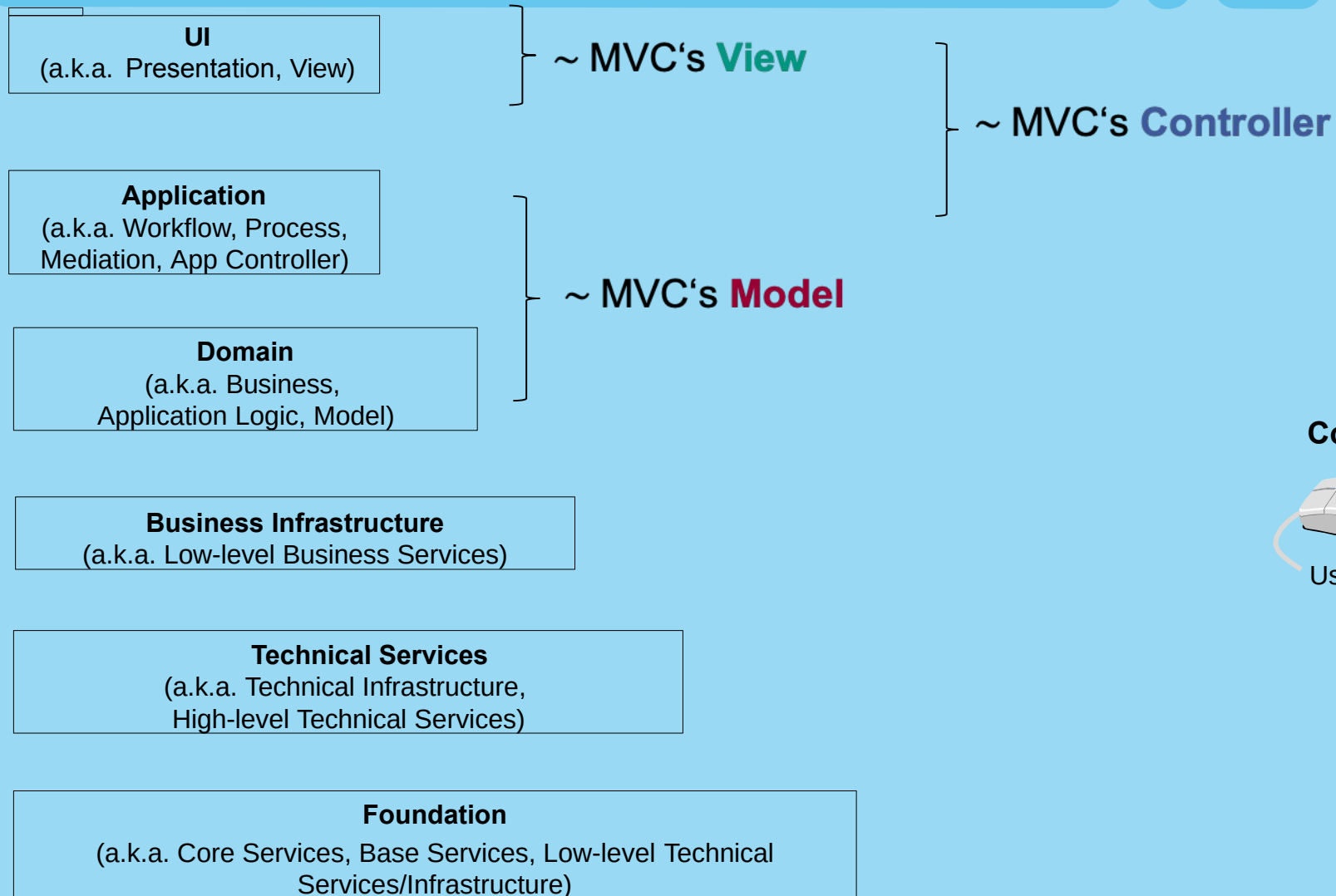
Class	Collaborators
Model	- View - Controller
Responsibility - Provides functional core of the application - Registers dependent views and controllers - Notifies dependent components about data changes	

Class	Collaborators
Controller	- View - Model
Responsibility - Accepts user input as events - Translates events to service requests for the model or display requests for the view - Implements the update procedure if required	



Class	Collaborators
View	- Controller - Model
Responsibility - Creates and initializes its associated controller - Displays information to the user - Implements the update procedure - Retrieves data from model	

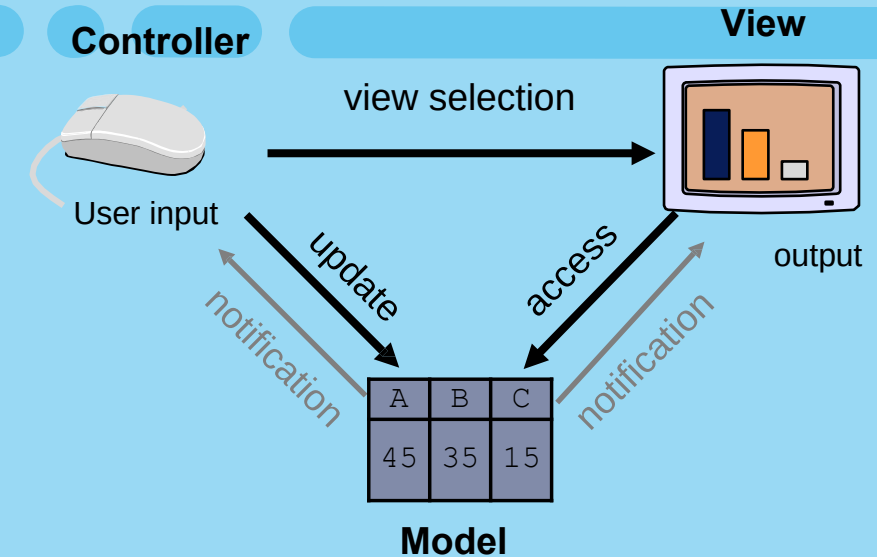
Layered Architecture and MVC



Model View Controller (MVC) Pattern (2/2)

Class	Collaborators
Model	- View - Controller
Responsibility - Provides functional core of the application - Registers dependent views and controllers - Notifies dependent components about data changes	

Class	Collaborators
Controller	- View - Model
Responsibility - Accepts user input as events - Translates events to service requests for the model or display requests for the view - Implements the update procedure if required	

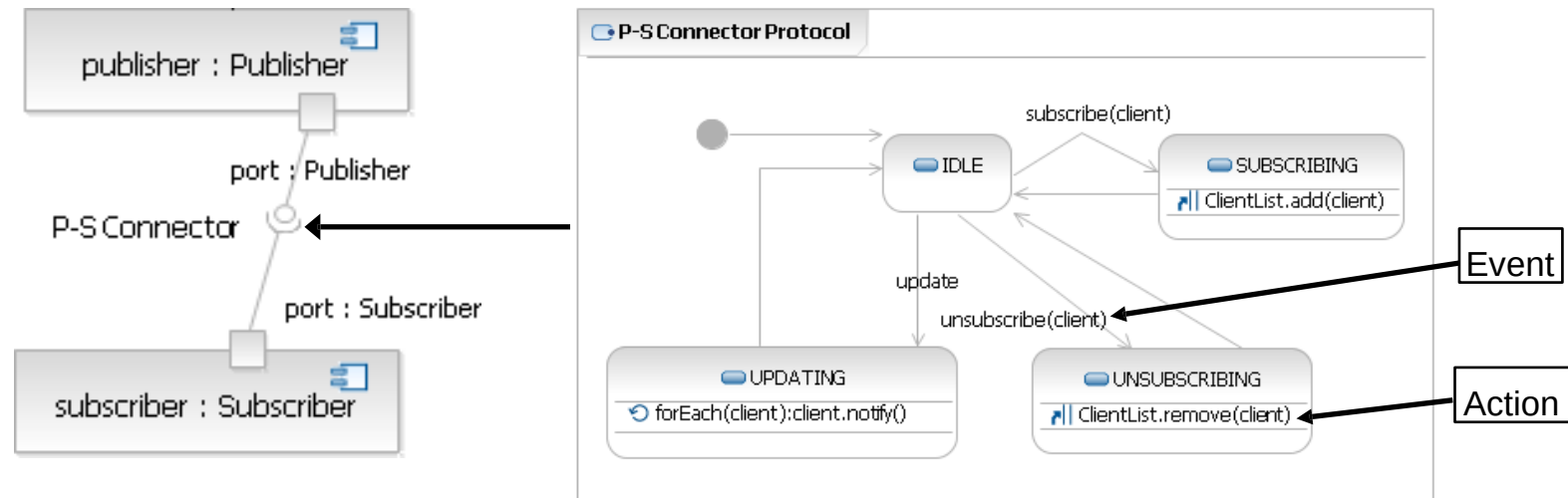


Class	Collaborators
View	- Controller - Model
Responsibility - Creates and initializes its associated controller - Displays information to the user - Implements the update procedure - Retrieves data from model	

Publish-Subscribe Architecture (1/2)

- Elements:
 - Components: components with interfaces for subscribing or publishing
 - Connectors: publish-subscribe
- Topology: bus (star)
- Examples: event messaging systems, chat, etc.
- Properties
 - Often the topology is dynamically changing (subscribe / unsubscribe during run time)
 - Supports loosely coupled systems (publisher decoupled from subscribers)

Publish-Subscribe Architecture (2/2)



Question

**Regarding Conway's law:
How might a publish-subscribe architecture fit within
an organizational framework?**

Concluding Remarks

- Architectural styles, architectural patterns, and reference architectures foster reuse of knowledge and best practices
- Apply Conway's Law to evaluate the applicability of architectural styles and patterns for your organization
- Good architectures strive to achieve –
 - cohesive responsibilities (high cohesion within architectural elements)
 - separation of concerns (low coupling between architectural elements)



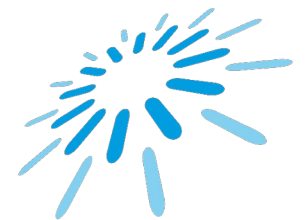
References

- [Larman05] C. Larman, Applying UML and Patterns (3rd ed.) Prentice Hall, 2005
- <https://www.udemy.com/course/software-architecture-and-design-essentials/>
- <https://martinfowler.com/microservices/>
- [Fowler03] M. Fowler, Patterns of Enterprise Architecture, Addison-Wesley, 2003
- [Endres/Rombach03] A. Endres, D. Rombach, A Handbook of Software and Systems Engineering, Addison-Wesley, 2003
- [GoF] E. Gamma, R. Helm, R. Johnson, J. Vlissides Design Patterns: Elements of Reusable OO Software Addison-Wesley, 1994
- S. Ambler, The Object Primer: Agile Model-Driven Development with UML 2.0 Cambridge University Press, 2004
- E. Evans, Domain-Driven Design Addison-Wesley, 2004



Until next time!

Software Architecture and Design Patterns
Prof. Dr. Jóakim von Kistowski



TH Aschaffenburg
university of applied sciences